

Phase 7

# Project Documentation Phase

Project: **Implement Client Script & UI Policy (Incident)**

| Item         | Details                                 |
|--------------|-----------------------------------------|
| Platform     | ServiceNow (Incident Management)        |
| Organization | SmartBridge                             |
| Team ID      | SWTID-2026-3598                         |
| Team Size    | 5                                       |
| Team Leader  | VEERAMANI B                             |
| Team Members | BALAJI E, Anbumani A, NAGULAN.C, Kathir |

# 1. Project Summary

| Item          | Details                                                                            |
|---------------|------------------------------------------------------------------------------------|
| Project title | Implement Client Script & UI Policy (Incident)                                     |
| Platform      | ServiceNow - Incident Management                                                   |
| Table         | Incident [incident]                                                                |
| Team ID       | SWTID-2026-3598                                                                    |
| Team Leader   | VEERAMANI B                                                                        |
| Team members  | BALAJI E, Anbumani A, NAGULAN.C, Kathir                                            |
| Skills        | Incident Management, UI Policy, UI Policy Actions, Client Scripts, Form Validation |

## 2. Problem and Objective

Incident data entered manually is often incomplete or inconsistent, harming triage, routing, reporting and SLA compliance. The project shows how UI Policies and Client Scripts enforce data integrity on Incident records by making fields mandatory, auto-populating values, controlling field behaviour and preventing invalid submissions.

## 3. Solution Components

| Component                                                       | Purpose                                                                 |
|-----------------------------------------------------------------|-------------------------------------------------------------------------|
| UI Policy 'High Impact Control'                                 | Applies extra controls when Impact = 1 - High; Reverse if false enabled |
| UI Policy Action - Assignment group                             | Mandatory = True, Read-only = False                                     |
| UI Policy Action - Urgency                                      | Read-only = True                                                        |
| Client Script - Auto set urgency for high impact (onChange)     | Sets Urgency to High and informs the user                               |
| Client Script - Prevent save if Assigned To missing (onSubmit)  | Blocks save with an error box                                           |
| Client Script - Prevent state change via list edit (onCellEdit) | Blocks inline State edits in lists                                      |

## 4. Configuration Reference

### 4.1 UI Policy

| Field                    | Value               |
|--------------------------|---------------------|
| Name / Short description | High Impact Control |
| Table                    | Incident            |
| Active                   | true                |

| Field            | Value              |
|------------------|--------------------|
| Condition        | Impact is 1 - High |
| Reverse if false | true               |

## 4.2 Client Scripts

| Name                                | Type       | Field  | Table    |
|-------------------------------------|------------|--------|----------|
| Auto set urgency for high impact    | onChange   | Impact | Incident |
| Prevent save if Assigned To missing | onSubmit   | -      | Incident |
| Prevent state change via list edit  | onCellEdit | State  | Incident |

## 4.3 Script Source

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }
    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage('Urgency set to High for High impact incident.');
```

```
    }
}

function onSubmit() {
    if (g_form.getValue('impact') == '1' &&
        g_form.getValue('assigned_to') == '') {
        g_form.showErrorMessage(
            'assigned_to',
            'Assigned To is mandatory for High impact incidents.'
        );
        return false;
    }
    return true;
}

function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    alert('State cannot be updated using list editing. Please open the Incident.');
```

```
    callback(false);
}
```

## 5. User Guide

- **Creating a High impact incident:** choose Impact = 1 - High. Urgency is set automatically and locked; select an Assignment group and an Assigned To user before submitting.
- **Lowering impact:** change Impact to Medium/Low; Assignment group stops being mandatory and Urgency becomes editable.
- **Changing State:** open the incident form; direct editing in the list is disabled.

## 6. Administrator Guide

- To disable a behaviour, set the relevant UI Policy or Client Script **Active** flag to false.

- To extend the rule to Impact = Medium, add an OR clause to the UI Policy condition and update the script checks.
- To allow list editing for specific roles, add a `g_user.hasRole()` check in the onCellEdit script.
- To enforce rules for API/imports, add an equivalent server-side Business Rule.

## 7. Troubleshooting

| Symptom                    | Likely cause                                         | Fix                                          |
|----------------------------|------------------------------------------------------|----------------------------------------------|
| Script does not run        | Inactive, wrong Type or Field name, UI Type mismatch | Check Active, Type, Field, UI Type = Desktop |
| Urgency not set            | Condition checks '1' but choice value differs        | Confirm choice values on the Impact field    |
| UI policy does not reverse | 'Reverse if false' unchecked                         | Enable the option and save                   |
| List edit still works      | onCellEdit script inactive or list not refreshed     | Activate script, reload list                 |

## 8. Results and Conclusion

The project effectively demonstrates how UI Policies and Client Scripts work together to enforce dynamic field behaviour, automate updates and prevent incorrect data submission on Incident forms. By using onChange, onSubmit and onCellEdit scripts alongside UI Policy actions, fields such as Assigned To and Urgency respond appropriately to changes in Impact, maintaining clean and consistent data. The solution is lightweight, efficient and easy to implement.

## 9. Future Enhancements

- Server-side Business Rule or Data Policy to enforce the same rules for APIs and imports.
- Extend rules to Category/Priority based conditions.
- Role-based exceptions for the State list-edit restriction.
- Automated testing with ServiceNow Automated Test Framework (ATF).